Predicting mutual interest before two strangers meet

Part 1 methods: data, predictors, preparation, models, metrics and checks

Math 195 capstone: Sophia Qin, Jenna Yang, Abraham Yeung

8 October 2026 (code at commit 1077096)

Abstract

This manual records every step that turns the raw Columbia speed-dating file into the Part 1 results: how the data were cleaned, every predictor (including the ones we constructed), how each training fold is prepared, exactly how each model (logistic regression, stepwise selection, LDA, QDA, random forest, lasso, the yes model and the liking model) is fitted and tuned, how AUC and the other metrics and their error bars are computed, and how each pre-registered hypothesis and robustness check is decided. It is generated by `scripts/methods_doc.py` from the code and data, so its lists and constants match what ran. The decisions themselves, with dates, are in `PREREG.md`.

Contents

1 The pipeline at a glance **2**

2 Data and unit of analysis **2**

2.1 The raw file 2

2.2 Cleaning (`clean.py`) 2

2.3 The pair table (`pairs.py`) 3

3 Predictors **4**

3.1 Each person's sign-up answers 4

3.2 The scorecard (in-date ratings) 4

3.3 Constructed pair features 4

3.4 Feature sets 7

4 Preparation before cross-validation **7**

5 Cross-validation **9**

5.1 Outer folds: whole events held out 9

5.2 Inner folds: tuning inside a training fold 9

6 Preparation inside each training fold **10**

7 The models **12**

7.1 Logistic regression (full model) . . . 13

7.2 Choosing splines by BIC . . . 14

7.3 Stepwise selection by BIC . . . 14

7.4 Linear discriminant analysis (LDA) . . . 15

7.5 Quadratic discriminant analysis (QDA) . . . 16

7.6 Random forest . . . 16

7.7 Lasso (L1-penalized logistic regression) . . . 16

7.8 The yes model (P5) . . . 17

7.9 The liking model (L1, L2) . . . 18

8 Metrics and error bars **19**

8.1 The metrics . . . 19

8.2 Cluster-robust error bars . . . 20

9 How each hypothesis is decided **21**

10 Robustness and sensitivity checks **22**

11 Diagnostics **22**

12 Simulation checks of the error bars **23**

13 Reproducing the results **23**

A Every predictor **24**

B Seeds **27**

C More code **27**

1 The pipeline at a glance

Run every script from the repository root, in this order. Each writes its outputs to `data/` or `results/` and prints the numbers the weekly report needs. Every random operation is seeded (Appendix B).

	Script (in scripts/)	What it does	Main output
1	`clean.py`	Cleans the raw file; one row per date from one side	`data/clean.csv`
2	`pairs.py`	One row per pair (dyad); constructs the pair features and feature sets	`data/pairs.csv`, `feature_sets.json`
3	`participants.py`	One row per participant (catalog)	`data/participants.csv`
4	`eda.py`	Exploratory tables and figures	`results/eda_*`
5	`data_description.py`	Report figures and captions	`results/data_description_*`
6	`sim_auc_cluster.py`	Checks the match-model error bars by simulation	`results/sim_auc_cluster_*`
7	`sim_like_cluster.py`	Checks the liking-model error bars	`results/sim_like_cluster_*`
8	`sim_coef_cluster.py`	Checks the coefficient and yes-model error bars	`results/sim_coef_cluster_*`
9	`like_models.py`	Liking model (L1, L2)	`results/like_models_*`
10	`part1_models.py`	All match models, 24 specifications, diagnostics, yes model (P1 to P5); about 30 minutes	`results/part1_models_*`
11	`part1_additions.py`	Pair-feature ladder (P3b, P3c), wave checks, P5 without wave 12	`results/part1_additions_*`
12	`fold_seed_noise.py`	Fold and forest seed noise (exploratory)	`results/fold_seed_noise*`
13	`compare_runs.py`	Earlier results next to the current ones	`results/compare_runs_*`

Tests with known answers live in `tests/` (run each file with `python tests/test_<name>.py`); every function that computes a variance, a fold assignment or a pair feature has one.

2 Data and unit of analysis

2.1 The raw file

The Kaggle release of the Columbia speed-dating experiment (Fisman, Iyengar, Kamenica and Simonson): 21 events (waves) between October 2002 and April 2004. Each person met every participant of the opposite sex at their event for four minutes, filled in a scorecard after each date (yes or no, six attribute ratings, overall liking, a guess of the partner's decision), and had filled in a sign-up survey before the event. Each row of the raw file is one date seen from one side. Always read alongside `data/speed_dating_key.doc`.

2.2 Cleaning (`clean.py`)

1. **Drop unusable rows.** 10 rows have no partner id: in wave 5, ten men each rated a woman (partner number 7) whose own record is missing from the release. No row lacks a decision.
2. **Identifiers.** `rater` (iid), `ratee` (pid), `event` (wave), `dyad` (the unordered pair) and `date_id` (the ordered pair).

3. **Stated preferences to shares of 100.** Three sign-up blocks ask how important each of six attributes is (own preferences `1_1`, what the opposite sex wants `2_1`, what peers want `4_1`). Waves 1–5 and 10–21 split 100 points; waves 6–9 rated each attribute from 1 to 10. Every block is rescaled to shares that sum to 100 (`<item>_share`); the survey format is kept in `pref_scale`.

4. **Blank allocations.** In the 100-point waves, where some items of a block were left blank but the answered items already sum to 100 (within 1 point), the blank items are set to 0, since no points were left to give (3 people, 40 dates). Each item left blank on the form is recorded in `<item>_was_blank`.

5. **Recording errors.** Gaming = 14 (5 people) and reading = 13 (3 people) recur in the same question across events on a 1–10 scale, so they are set to missing; the recorded values are kept in `gaming_recorded` and `reading_recorded` for the robustness grid. Ratings of 0 are kept as deliberate answers.

6. **Checks**, all zero: partners in a different wave, `match` $\neq$ both decisions, duplicate dates.

2.3 The pair table (`pairs.py`)

Match prediction uses one row per dyad: one woman and one man who met (4,184 dyads, 690 matches, 16.5%). The response is `match` = 1 when both said yes. Each person's own answers appear with prefix `f_` (the woman) or `m_` (the man); for the scorecard columns the prefix says whose view it is (`f_rating_attr` is her rating of his attractiveness). The two decisions `f_dec` and `m_dec` define the response and are never predictors. A check rebuilds the table with the genders swapped and confirms every symmetric pair feature is unchanged. The 551 participants each appear in many dyads, which is why the error bars treat events as clusters (Section 8).

3 Predictors

3.1 Each person's sign-up answers

Column (p = f or m)	Question	Scale
p_age	Age	years
p_field_cd	Field of study (18 codes; Appendix A)	categorical
p_career_c	Intended career (17 codes)	categorical
p_goal	Primary goal in coming (6 codes)	categorical
p_date	How often do you go on dates?	1–7, 1 = most often
p_go_out	How often do you go out?	1–7, 1 = most often
p_exphappy	How happy do you expect to be with the people you meet?	1–10
p_<interest>	Interest in each of 17 activities: playing sports/athletics, watching sports, body building/exercising, dining out, museums/galleries, art, hiking/camping, gaming, dancing/clubbing, reading, watching tv, theater, movies, going to concerts, music, shopping, yoga/meditation	1–10
p_self_<a>	Self-rating on five attributes: attractive, sincere, intelligent, fun, ambitious	1–10
p_pref_<a>	Stated importance of six attributes (the five plus shared interests), as shares of 100	share

Each appears twice, once for the woman and once for the man. The share for shared interests is left out of every model (pref_reference): the six shares sum to 100, so one is redundant.

3.2 The scorecard (in-date ratings)

Used only in the pre+date feature set. For each side p: p_rating_<a> for the six attributes (1 = awful, 10 = great), p_rating_like (overall, how much do you like this person; 1–10) and p_prob (how probable is it that this person will say yes to you; 1–10).

3.3 Constructed pair features

These are new columns we built from the two people's answers. The first six are the pair features of P3; the next five are the extended features of P3b; the last three are the declared terms of P3c. All use only sign-up answers. Where a definition needs an answer that is missing, the feature is missing and is filled inside each training fold (Section 6).

Feature	Definition	Set	Missing
abs_age_diff	Age gap |man's age - woman's age|	P3	94
int_corr	Interest similarity: Pearson correlation of the two people's ratings of the 16 interests other than yoga	P3	79
same_field	1 if the two field codes are equal	P3	82
same_goal	1 if the two goal codes are equal	P3	79
fit_to_f_prefs	How well the man's relative strengths fit the woman's priorities (compatibility term, equation 1)	P3	97
fit_to_m_prefs	How well the woman's relative strengths fit the man's priorities	P3	87
same_career	1 if the two career codes are equal	P3b	137
abs_date_diff	Gap in dating frequency |f_date - m_date|	P3b	97
abs_go_out_diff	Gap in going-out frequency |f_go_out - m_go_out|	P3b	79
int_mad	Mean absolute difference of the 17 interest ratings	P3b	79
int_cosine	Cosine similarity of the two 17-interest vectors	P3b	79
age_diff_sq	Squared signed age difference (m_age - f_age)^2	P3c	–
int_x_f_pref_shar	Interest similarity times the woman's (standardized) weight on shared interests	P3c	–
int_x_m_pref_shar	Interest similarity times the man's (standardized) weight on shared interests	P3c	–

Missing = dyads where it cannot be computed (of 4,184). The P3c terms are built after the survey-format standardization (Section 4), so they use the standardized shares.

Compatibility (preference fit). With w_a the woman's stated share for attribute a, rescaled to sum to 1 over the five self-rated attributes (shared interests has no self-rating), s_a the man's self-rating and $\bar{s}$ his mean self-rating over the five,

$$\texttt{fit_to_f_prefs} = \sum_{a=1}^{5} \left(w_a - \tfrac{1}{5}\right)(s_a - \bar{s}). \tag{1}$$

It is positive when the attributes she weights above average are the ones he rates himself relatively highest on, and 0 when either gave all-equal answers. The plain weighted mean $\sum_a w_a s_a$ (option A, kept for the robustness grid as `fit_raw_to_f_prefs`) equals $\bar{s}$ plus this term: it is 94% his overall self-rating, a fact about him alone, so the headline uses only the compatibility term (option B). `fit_to_m_prefs` swaps the roles.

Interest similarity. `int_corr` is the Pearson correlation of the two people's ratings of the 16 interests other than yoga, over the interests both answered (at least 3), unrounded and after the recording-error fix. This reproduces the released `int_corr` exactly (rounded to 2 decimals) wherever no fix applies. `int_mad` and `int_cosine` use all 17 interests, over those both answered (at least 3):

$$\texttt{int_mad} = \frac{1}{|S|}\sum_{i\in S}|f_i - m_i|, \qquad \texttt{int_cosine} = \frac{\sum_{i\in S} f_i m_i}{\sqrt{\sum_{i\in S} f_i^2 \sum_{i\in S} m_i^2}}.$$

Missing-answer flags. `f_pre_missing` and `m_pre_missing` are 1 when that person left any sign-up answer blank on the form (recorded values count, not the recording-error fix, and so do preference items the blank-allocation rule set to 0). `f_date_missing` and `m_date_missing` are 1 when that person left any scorecard rating blank. They let a model learn whether leaving answers blank itself predicts a match (pairs with a blank in-date answer match less often).

The pair features are built in `pairs.build_pairs`, with the helper functions below.

Code 1: The constructed pair features and flags (scripts/pairs.py)

```
    new = pd.DataFrame({
        "age_diff": age_diff,
        "abs_age_diff": age_diff.abs(),
        "int_corr": row_corr_pairwise(cols("f", INTERESTS_INT_CORR), cols("m", INTERESTS_INT_CORR)),
        "int_corr17": row_corr_pairwise(cols("f", INTERESTS), cols("m", INTERESTS)),
        "same_field": same_code(pairs.f_field_cd, pairs.m_field_cd),
        "same_goal": same_code(pairs.f_goal, pairs.m_goal),
        "same_career": same_code(pairs.f_career_c, pairs.m_career_c),
        "abs_date_diff": (pairs.f_date - pairs.m_date).abs(),
        "abs_go_out_diff": (pairs.f_go_out - pairs.m_go_out).abs(),
        "int_mad": row_mad_pairwise(cols("f", INTERESTS), cols("m", INTERESTS)),
        "int_cosine": row_cosine_pairwise(cols("f", INTERESTS), cols("m", INTERESTS)),
        "fit_to_f_prefs": compat_fit(prefs("f"), selfs("m")),
        "fit_to_m_prefs": compat_fit(prefs("m"), selfs("f")),
        "fit_raw_to_f_prefs": pref_fit(prefs("f"), selfs("m")),
        "fit_raw_to_m_prefs": pref_fit(prefs("m"), selfs("f")),
        **{f"{p}_pre_missing": (pairs[recorded_person_cols(p)].isna().any(axis=1)
                                | pairs[[f"{p}_{v}" for v in WAS_BLANK.values()]].eq(1).any(axis=1)).astype(int)
      for p in "fm"},
        **{f"{p}_date_missing": cols(p, IN_DATE.values()).isna().any(axis=1).astype(int) for p in "fm"},
    }, index=pairs.index)
```

Code 2: Compatibility term, equation (1) (scripts/pairs.py)

```
def compat_fit(weights, self_ratings):
    """Option B (PREREG.md): sum over the k attributes of (weight share - 1/k) x (self-rating - own mean self-
     rating),
    weights rescaled to sum to 1. Equals pref_fit minus the mean self-rating. NaN if any value in the row is
     missing."""
    W, S = np.asarray(weights, float), np.asarray(self_ratings, float)
    with np.errstate(invalid="ignore", divide="ignore"):
        P = W / W.sum(axis=1, keepdims=True)
    return ((P - 1 / W.shape[1]) * (S - S.mean(axis=1, keepdims=True))).sum(axis=1)
```

Code 3: Interest similarity over the interests both answered (scripts/pairs.py)

```
def row_corr_pairwise(A, B, min_n=3):
    """Pearson correlation between matching rows of A and B over the columns where both are present.
    NaN when fewer than min_n columns are shared or either side has no spread there."""
    A, B = np.asarray(A, float), np.asarray(B, float)
    ok = ~(np.isnan(A) | np.isnan(B))
    n = ok.sum(axis=1)
    with np.errstate(invalid="ignore", divide="ignore"):
        ma = np.where(ok, A, 0.0).sum(axis=1) / n
        mb = np.where(ok, B, 0.0).sum(axis=1) / n
        ac, bc = np.where(ok, A - ma[:, None], 0.0), np.where(ok, B - mb[:, None], 0.0)
        r = (ac * bc).sum(axis=1) / np.sqrt((ac ** 2).sum(axis=1) * (bc ** 2).sum(axis=1))
    r[n < min_n] = np.nan
    return r
```

Code 4: Mean absolute difference of interests (scripts/pairs.py)

```
def row_mad_pairwise(A, B, min_n=3):
    """Mean absolute difference between matching rows over the columns both have; NaN with fewer than min_n."""
    A, B = np.asarray(A, float), np.asarray(B, float)
    ok = ~(np.isnan(A) | np.isnan(B)); n = ok.sum(axis=1)
    with np.errstate(invalid="ignore", divide="ignore"):
        r = np.where(ok, np.abs(A - B), 0.0).sum(axis=1) / n
    r[n < min_n] = np.nan
    return r
```

Code 5: Cosine similarity of interests (scripts/pairs.py)

```
def row_cosine_pairwise(A, B, min_n=3):
    """Cosine similarity between matching rows over the columns both have; NaN with fewer than min_n or a zero
     row."""
```

```
    A, B = np.asarray(A, float), np.asarray(B, float)
    ok = ~(np.isnan(A) | np.isnan(B)); n = ok.sum(axis=1)
    a, b = np.where(ok, A, 0.0), np.where(ok, B, 0.0)
    with np.errstate(invalid="ignore", divide="ignore"):
        r = (a * b).sum(axis=1) / np.sqrt((a ** 2).sum(axis=1) * (b ** 2).sum(axis=1))
    r[n < min_n] = np.nan
    return r
```

Code 6: Same field, goal or career (scripts/pairs.py)

```
def same_code(a, b):
    """1.0 where two code columns agree, 0.0 where they differ, NaN where either is missing."""
    a, b = np.asarray(a, float), np.asarray(b, float)
    out = (a == b).astype(float)
    out[np.isnan(a) | np.isnan(b)] = np.nan
    return out
```

3.4 Feature sets

Set	Contents	Raw columns
`pre`	Everything known before the date: both people's sign-up answers and the six pair features	76 (74 used)
`pre+date`	Adds the 16 scorecard columns	92 (90 used)
`pre_individual`	Both people's sign-up answers only (for P3)	70 (68 used)
`pre_extended`	`pre` plus the five P3b features	79 used
`pre_interactions`	`pre_extended` plus the three P3c terms	82 used

"Used" drops the two shared-interests shares. The missing-answer flags are added on top (the two sign-up flags; for `pre+date` also the two scorecard flags), except in the complete-case version.

Never used as predictors: race and race-preference variables (a pre-registered decision), `f_dec` and `m_dec` (they define the response), the signed `age_diff` (a linear combination of the two ages), `int_corr17` (a near-duplicate of `int_corr`), the event-level flags, and the order of the date in the evening.

4 Preparation before cross-validation

Two steps use no outcomes. They are applied to the whole pair table, in `modeling.build_frame`, before any fold is formed.

1. **Survey-format standardization.** Waves 6–9 rated each attribute 1–10, so their converted shares are flat (a person's six shares have SD 3.4 against 9.2 in the 100-point waves). Each stated-preference share and the two compatibility terms are rescaled to mean 0 and SD 1 within each format:

$$z = \frac{x - \mu_{\text{format}}}{\sigma_{\text{format}}},$$

   where for the shares μ and σ come from one row per participant (women and men pooled) in that format, and for the compatibility terms from one row per dyad in that format. Because only survey answers are used, this stays valid even when a fold holds out every waves 6–9 event at once.

2. **Specification choices** (Section 10): which interest values (fixed or as recorded), which preference fit (B or A), whether to standardize by format, and how to treat missing values.

Code 7: Survey-format standardization (scripts/modeling.py)

```
def standardize_by_format(df, person_cols, dyad_cols, fmt_col="pref_scale"):
    """Rescale to mean 0, SD 1 within each survey format. Person columns (f_x and m_x pairs) use one row per
    participant, pooling women and men; dyad columns use one row per dyad. Uses no outcomes."""
    out = df.copy()
    for col in person_cols:                    # e.g. f_pref_attr; its partner column is m_pref_attr
        stem = col[2:]
        people = pd.concat([df[["f_iid", fmt_col, f"f_{stem}"]].drop_duplicates("f_iid").set_axis(["iid", "fmt",
      "v"], axis=1),
                            df[["m_iid", fmt_col, f"m_{stem}"]].drop_duplicates("m_iid").set_axis(["iid", "fmt",
      "v"], axis=1)])
        stats = people.groupby("fmt").v.agg(["mean", "std"])
        mu, sd = df[fmt_col].map(stats["mean"]), df[fmt_col].map(stats["std"])
        out[col] = (df[col] - mu) / sd
    for col in dyad_cols:
        stats = df.groupby(fmt_col)[col].agg(["mean", "std"])
        out[col] = (df[col] - df[fmt_col].map(stats["mean"])) / df[fmt_col].map(stats["std"])
    return out
```

Code 8: The raw design for one specification and feature set (scripts/modeling.py)

```
def build_frame(pairs, sets, spec, feature_set, exclude_events=(), complete_extra=()):
    """Raw design for one specification and feature set: 'pre', 'pre+date', 'pre_individual', or the P3b/P3c
    ladder steps 'pre_extended' (plus the five extended pair features) and 'pre_interactions' (plus the three
    declared terms). exclude_events drops whole waves first (the waves 6 to 9 and wave 12 checks). Under the
    complete-case specification a dyad must be complete on every column of `pre+date` the models use, plus any
    complete_extra columns (the complete-case ladder passes the shared-interests shares, which P3c's terms use).
    Returns (X, y, events, ids) where ids holds f_iid and m_iid; columns keep their canonical names."""
    df = pairs[~pairs.event.isin(exclude_events)].copy()
    subs = {}
    if spec.out_of_range == "recorded":
        subs.update(sets["alternatives"]["out_of_range_as_recorded"])
    if spec.fit == "A":
        subs.update(sets["alternatives"]["preference_fit_A"])
    for canon, alt in subs.items():
        df[canon] = df[alt]
    if spec.pref_format == "standardized":
        person = [c for c in sets["format_standardized"] if c.startswith("f_pref_")]
        person += ["m" + c[1:] for c in person]
        dyad = [c for c in sets["format_standardized"] if not c.startswith(("f_pref_", "m_pref_"))]
        df = standardize_by_format(df, person, dyad)
    df["age_diff_sq"] = (df.m_age - df.f_age) ** 2
    df["int_x_f_pref_shar"] = df.int_corr * df.f_pref_shar
    df["int_x_m_pref_shar"] = df.int_corr * df.m_pref_shar
    cols = {"pre": sets["pre"], "pre+date": sets["pre+date"], "pre_individual": sets["pre_individual"],
            "pre_extended": sets["pre"] + sets.get("extended_pair", []),
            "pre_interactions": sets["pre"] + sets.get("extended_pair", []) + INTERACTIONS}[feature_set]
    cols = [c for c in cols if c not in sets["pref_reference"]]
    if spec.missing == "complete":
        full = [c for c in sets["pre+date"] if c not in sets["pref_reference"]] + list(complete_extra)
        df = df[df[full].notna().all(axis=1)]
    else:
        cols = cols + sets["flags_pre"]
        if spec.missing == "median_flags" and feature_set == "pre+date":
            cols = cols + sets["flags_date"]
    return (df[cols].reset_index(drop=True), df["match"].to_numpy(float), df["event"].to_numpy(int),
            df[["f_iid", "m_iid"]].reset_index(drop=True))
```

5 Cross-validation

5.1 Outer folds: whole events held out

The 21 events are put in a random order (seed 20261006) and dealt into 5 folds in turn. Each fold holds out its events; models are fitted on the other four folds and score the held-out pairs. All five folds' held-out predictions are pooled, and every metric is computed once on the pooled set. Holding out whole events means a model is never scored on a person it saw in training: random row splits would put the same people on both sides and overstate accuracy.

Fold	Events held out	Dyads	Matches
0	5, 10, 15, 17, 21	1,137	201
1	8, 12, 13, 16	434	67
2	2, 14, 18, 19	925	134
3	1, 4, 6, 9	849	163
4	3, 7, 11, 20	839	125

Threshold. A pair is predicted to match when its predicted probability exceeds the training fold's match rate (about 0.165), chosen on training data only. This puts the cut where an uninformed model would sit, since matches are rare; AUC does not depend on it.

Code 9: Outer event folds (scripts/modeling.py)

```
def event_folds(events, k=5, seed=FOLD_SEED):
    """Map each event to a fold 0..k-1: a seeded random order of the events, dealt out in turn."""
    ev = np.sort(np.unique(events))
    order = np.random.default_rng(seed).permutation(len(ev))
    return {int(ev[i]): int(j % k) for j, i in enumerate(order)}
```

5.2 Inner folds: tuning inside a training fold

QDA, the random forest and the lasso each have one setting to tune. Inside outer training fold k, the training events are dealt into 4 inner folds (seed 20261006 + 1 + k). For each grid value, the model is fitted on three inner folds (with the preparation of Section 6 refitted on them) and predicts the fourth; the four inner held-out sets are pooled and the grid value with the highest pooled AUC is chosen (ties go to the earlier value). The model is then refitted on the whole training fold with that value. The outer test fold is never touched.

Code 10: Inner tuning by pooled AUC (scripts/part1_models.py)

```
def tune_inner(Xtr, ytr, evtr, idtr, sets, k, make_model, grid):
    """The grid value with the best pooled inner AUC over event-grouped 4-fold CV on the training fold
    (preparation refitted on each inner training part; ties go to the earlier grid value)."""
    inner = M.event_folds(evtr, k=4, seed=M.FOLD_SEED + 1 + k)
    f = np.array([inner[e] for e in evtr])
    probs = {r: np.empty(len(ytr)) for r in grid}
    for j in range(4):
        a, b = f != j, f == j
        p = make_prep(sets, Xtr[a]).fit(Xtr[a].reset_index(drop=True), idtr[a].reset_index(drop=True))
        Da, Db = p.transform(Xtr[a].reset_index(drop=True)).to_numpy(), p.transform(Xtr[b].reset_index(drop=True
    )).to_numpy()
        for r in grid:
            probs[r][b] = predict_serial(make_model(r).fit(Da, ytr[a]), Db)
    return max(grid, key=lambda r: M.auc(probs[r], ytr))
```

6 Preparation inside each training fold

Every quantity below is estimated on the training fold only and then applied unchanged to the held-out events (class `modeling.Prep`), in this order.

1. **Skewness rule.** For each numeric column with more than two distinct values, the sample skewness $\gamma = \frac{1}{n}\sum_i \left((x_i - \bar{x})/s\right)^3$ is computed on the training fold (over one value per participant for a person's own answers, over dyads for pair features). If $\gamma > 1$ the column becomes $\log(1 + x - \min x)$; if $\gamma < -1$ it becomes $-\log(1 + \max x - x)$, with the minimum and maximum from the training fold and held-out values clipped to that range first. Binary columns are never transformed.

2. **Missing values.** Each numeric column's missing values are filled with its training-fold median (after the transform); binary columns with their training median. Under the headline specification the missing-answer flags (Section 3) are included as predictors, so the model can tell a filled value from an answered one.

3. **Categorical codes.** Field, career and goal are categorical. A field or career code held by fewer than 10 participants in the training fold (each person counted once) is merged into "other"; a missing code is its own level "missing". The most common level in the training fold is the reference; every other level gets a 0/1 dummy column. A held-out code never seen in training goes to "other" (or the reference if there is no "other").

4. **Splines** (logistic and stepwise only, Section 7.2). A natural cubic spline with knots at the training-fold quantiles $0, \frac{1}{3}, \frac{2}{3}, 1$ of the filled column ($K = 4$ knots, 3 degrees of freedom) replaces a candidate's straight line. With $d_k(x) = \left[(x - \xi_k)_+^3 - (x - \xi_K)_+^3\right]/(\xi_K - \xi_k)$, the basis is x and $N_{k+2}(x) = d_k(x) - d_{K-1}(x)$ for $k = 1, \ldots, K-2$ (Hastie, Tibshirani and Friedman, *ESL*, 5.2.1).

5. **Standardizing.** Every numeric column, spline columns included, is rescaled to the training fold's mean 0 and SD 1. Dummies and flags stay 0/1.

6. **Identical columns.** When two columns are identical in a training fold (for example two missing-code dummies held by the same people) both are kept: the data cannot divide their combined effect, and the small ridge in the logistic fit divides it equally.

Code 11: Fitting the preparation on a training fold (scripts/modeling.py)

```
    def fit(self, X, ids):
        self.numeric = [c for c in X.columns if c not in self.categorical and c not in self.binary]
        self.bins = [c for c in X.columns if c in self.binary]
        self.tf = {}
        for c in self.numeric:                                  # skewness rule
            v = X[c].to_numpy(float)
            if X[c].nunique(dropna=True) <= 2:
                continue
            if c in self.person_cols:                           # one value per participant
                side = c.split("_")[0]
                v = pd.concat([X[c], ids[f"{side}_iid"]], axis=1).drop_duplicates(f"{side}_iid")[c].to_numpy(
    float)
            s = skew(v)
            if s > SKEW_LIMIT:
                self.tf[c] = ("right", np.nanmin(X[c]), np.nanmax(X[c]))
            elif s < -SKEW_LIMIT:
                self.tf[c] = ("left", np.nanmin(X[c]), np.nanmax(X[c]))
        T = self._transform_skew(X)
        self.medians = {c: float(np.nanmedian(T[c])) for c in self.numeric}
        self.bin_medians = {c: float(np.nanmedian(X[c])) if X[c].notna().any() else 0.0 for c in self.bins}
        self.levels = {}
```

```
    for c in self.categorical:                                  # rare codes, then one-hot levels
        codes = X[c]
        if any(c.endswith(m) for m in self.merge_codes):
            stem = c.split("_", 1)[1]
            held = (pd.concat([pd.DataFrame({"iid": ids[f"{s}_iid"].to_numpy(), "code": X[f"{s}_{stem}"].
 to_numpy()})
                               for s in self.sides if f"{s}_{stem}" in X.columns])
                    .drop_duplicates("iid").code.value_counts())       # each participant counted once
            rare = set(held.index[held < RARE_CODE_MIN])
        else:
            rare = set()
        lab = self._labels(codes, rare)
        counts = lab.value_counts()
        self.levels[c] = (rare, [l for l in counts.index if l != counts.index[0]], counts.index[0])
    F = self._filled(T)
    self.knots = {}
    for c in self.splines:
        if c in F.columns:
            q = np.unique(np.quantile(F[c], [0, 1 / 3, 2 / 3, 1]))
            if len(q) == 4:
                self.knots[c] = q
    D = self._design(F, X)
    # Columns identical in the training fold (e.g. two missing-code dummies held by the same people) are all
    # kept: the data cannot divide their combined effect, and fit_logit's ridge divides it equally (PREREG.md,
    # second set of fixes of 2026-10-08).
    self.mu = {c: D[c].mean() for c in D.columns if not c.startswith(("cat:", "bin:"))}
    self.sd = {c: (D[c].std() or 1.0) for c in self.mu}
    self.columns = list(D.columns)
    return self

def _transform_skew(self, X):
    T = X.copy()
    for c, (kind, lo, hi) in self.tf.items():
        v = T[c].clip(lo, hi)
        T[c] = np.log1p(v - lo) if kind == "right" else -np.log1p(hi - v)
    return T
```

Code 12: Building and applying the design (scripts/modeling.py)

```
def _labels(codes, rare):
    lab = codes.map(lambda v: "missing" if pd.isna(v) else ("other" if v in rare else str(int(v))))
    return lab

def _filled(self, T):
    F = T.copy()
    for c in self.numeric:
        F[c] = F[c].fillna(self.medians[c])
    return F

def _design(self, F, X):
    parts = {}
    for c in self.numeric:
        if c in self.knots:
            B = natural_spline_basis(F[c], self.knots[c])
            for j in range(B.shape[1]):
                parts[f"{c}" if j == 0 else f"{c}:ns{j}"] = B[:, j]
        else:
            parts[c] = F[c].to_numpy(float)
    for c in self.bins:
        parts[f"bin:{c}"] = X[c].fillna(self.bin_medians[c]).to_numpy(float)
    for c in self.categorical:
        rare, keep, _ = self.levels[c]
        lab = self._labels(X[c], rare)
        known = set(keep) | {self.levels[c][2]}
        lab = lab.where(lab.isin(known), "other" if "other" in known else self.levels[c][2])
        for l in keep:
            parts[f"cat:{c}={l}"] = (lab == l).to_numpy(float)
    return pd.DataFrame(parts)
```

```
    def transform(self, X):
        D = self._design(self._filled(self._transform_skew(X)), X)
        for c in self.columns:
            if c not in D.columns:
                D[c] = 0.0
        D = D[self.columns]
        for c in self.mu:
            D[c] = (D[c] - self.mu[c]) / self.sd[c]
        return D
```

Code 13: Natural cubic spline basis (scripts/modeling.py)

```
def natural_spline_basis(x, knots):
    """Natural cubic spline basis without the constant (ESL 5.2.1): x, then N_{k+2} = d_k - d_{K-1}, k = 1..K-2.
    """
    x = np.asarray(x, float); K = len(knots)
    d = lambda k: (np.clip(x - knots[k], 0, None) ** 3 - np.clip(x - knots[-1], 0, None) ** 3) / (knots[-1] -
     knots[k])
    return np.column_stack([x] + [d(k) - d(K - 2) for k in range(K - 2)])
```

The prepared design has 124 to 128 columns for `pre` and 142 to 146 for `pre+date` under the headline specification, depending on the fold (dummies and splines differ by fold).

7 The models

Each model below is fitted on the prepared training fold and predicts a match probability for each held-out pair. Logistic regression and stepwise selection use the design with the splines chosen in that fold; LDA, QDA, the random forest and the lasso use the same preparation without splines (they can model curvature themselves or are linear by design).

Code 14: One outer fold: preparation and every model (scripts/part1_models.py)

```
def run_fold(k, X, y, ev, ids, sets, folds, models, rf_seed=RF_SEED):
    import warnings; warnings.filterwarnings("ignore")
    test = np.array([folds[e] == k for e in ev]); train = ~test
    Xtr, Xte, ytr = X[train].reset_index(drop=True), X[test].reset_index(drop=True), y[train]
    idtr = ids[train].reset_index(drop=True)
    out, info = {}, dict(fold=k, threshold=float(ytr.mean()), n_train=int(train.sum()), n_test=int(test.sum()),
                         inner_seed=M.FOLD_SEED + 1 + k, rf_random_state=rf_seed + k, lasso_random_state=
     LASSO_SEED + k)
    spl = choose_splines(Xtr, ytr, idtr, sets)
    p = make_prep(sets, Xtr, splines=spl).fit(Xtr, idtr)
    Dtr, Dte = p.transform(Xtr).to_numpy(), p.transform(Xte).to_numpy()
    w, _ = M.fit_logit(Dtr, ytr)
    out["logistic"] = M.predict_logit(w, Dte)
    info["splines"] = ";".join(spl); info["columns"] = Dtr.shape[1]
    sel = {}
    if "stepwise" in models:
        blocks = p.blocks()
        for d in DIRECTIONS:
            chosen = M.stepwise(Dtr, ytr, blocks, d)
            cols = [j for b in chosen for j in blocks[b]]
            wd, _ = M.fit_logit(Dtr[:, cols], ytr)
            out[f"stepwise_{d}"] = M.predict_logit(wd, Dte[:, cols])
            sel[d] = chosen
    if "lda_qda" in models or "rf" in models or "lasso" in models:
        pl = make_prep(sets, Xtr).fit(Xtr, idtr)
        Ltr, Lte = pl.transform(Xtr).to_numpy(), pl.transform(Xte).to_numpy()
    if "lda_qda" in models:
        out["lda"] = LinearDiscriminantAnalysis().fit(Ltr, ytr).predict_proba(Lte)[:, 1]
        reg = tune_inner(Xtr, ytr, ev[train], idtr, sets, k, make_qda, QDA_REGS)
        out["qda"] = make_qda(reg).fit(Ltr, ytr).predict_proba(Lte)[:, 1]
        info["qda_reg_param"] = reg
```

```
    if "rf" in models:
        leaf = tune_inner(Xtr, ytr, ev[train], idtr, sets, k, make_rf(k, rf_seed), RF_LEAVES)
        out["rf"] = predict_serial(make_rf(k, rf_seed)(leaf).fit(Ltr, ytr), Lte)
        info["rf_min_samples_leaf"] = leaf
    if "lasso" in models:
        C = tune_inner(Xtr, ytr, ev[train], idtr, sets, k, make_lasso(k), lasso_grid(Ltr, ytr))
        las = make_lasso(k)(C).fit(Ltr, ytr)
        out["lasso"] = las.predict_proba(Lte)[:, 1]
        nz = np.abs(las.coef_[0]) > 1e-8
        sel["lasso"] = [b for b, cols in pl.blocks().items() if nz[cols].any()]
        info["lasso_C"] = C; info["lasso_nonzero_columns"] = int(nz.sum())
    return dict(test_idx=np.where(test)[0], prob=out, info=info, selected=sel)
```

Code 15: Event cross-validation (scripts/part1_models.py)

```
def cross_validate(pairs, sets, spec, feature_set, models, jobs, exclude_events=(), complete_extra=(), folds=
    None,
                   fold_seed=M.FOLD_SEED, rf_seed=RF_SEED):
    """Event 5-fold CV of the given models. Folds: by default the events used are dealt into 5 folds by
    fold_seed;
    `folds` (event -> fold) fixes them instead, e.g. the headline folds with some waves left out."""
    X, y, ev, ids = M.build_frame(pairs, sets, spec, feature_set, exclude_events, complete_extra)
    folds_used = M.event_folds(ev, seed=fold_seed) if folds is None else {e: folds[e] for e in np.unique(ev)}
    fold_label = f"seed {fold_seed} over the events used" if folds is None else "given (headline folds)"
    res = Parallel(n_jobs=jobs)(delayed(run_fold)(k, X, y, ev, ids, sets, folds_used, models, rf_seed) for k in
    range(5))
    names = list(res[0]["prob"])
    prob = {m: np.empty(len(y)) for m in names}; thr = np.empty(len(y))
    for r in res:
        for m in names:
            prob[m][r["test_idx"]] = r["prob"][m]
        thr[r["test_idx"]] = r["info"]["threshold"]
    rows = []
    for m in names:
        met = M.metric_table(prob[m], y, thr, ev)
        rows.append(dict(spec=spec.label, missing=spec.missing, fit=spec.fit, out_of_range=spec.out_of_range,
                         pref_format=spec.pref_format, headline=spec == M.HEADLINE, feature_set=feature_set,
    model=m,
                         fold_scheme="event", folds=fold_label, dyads=len(y), matches=int(y.sum()), seed=
    fold_seed,
                         rf_seed=rf_seed, lasso_seed=LASSO_SEED,
                         **{f"{k}": v[0] for k, v in met.items()}, **{f"{k}_se": v[1] for k, v in met.items()}))
    fold_info = [dict(spec=spec.label, feature_set=feature_set, **r["info"]) for r in res]
    selection = [dict(spec=spec.label, feature_set=feature_set, direction=d, fold=r["info"]["fold"], block=b)
                 for r in res for d, bl in r["selected"].items() for b in bl]
    return rows, fold_info, selection, (prob, y, ev, X)
```

7.1 Logistic regression (full model)

The probability of a match is $p_i = 1/(1 + e^{-\eta_i})$ with $\eta_i = \beta_0 + x_i^\top \beta$. The coefficients maximize the log-likelihood with a tiny ridge that keeps separated fits finite,

$$\ell(\beta) - \tfrac{\lambda}{2} \sum_{j \geq 1} \beta_j^2, \qquad \ell(\beta) = \sum_i \left[y_i \eta_i - \log(1 + e^{\eta_i}) \right], \qquad \lambda = 10^{-6},$$

by iteratively reweighted least squares (Newton's method): with $W = \mathrm{diag}\{p_i(1 - p_i)\}$ and $\Lambda = \mathrm{diag}(0, \lambda, \ldots, \lambda)$,

$$\beta \leftarrow \beta + (X^\top W X + \Lambda)^{-1} (X^\top (y - p) - \Lambda \beta),$$

stopping when no coefficient moves by more than 10^{-8} (`modeling.fit_logit`). The full model uses every column of the feature set; with 690 matches and more than 120 columns it overfits, which is why stepwise selection and the regularized models exist.

Code 16: Logistic regression by IRLS (scripts/modeling.py)

```
def fit_logit(X, y, ridge=1e-6, iters=60, w0=None):
    """Logistic regression with intercept by IRLS (a tiny ridge keeps separated fits finite).
    Returns (coefficients with intercept first, log-likelihood)."""
    n, p = X.shape
    Xb = np.hstack([np.ones((n, 1)), X])
    w = np.zeros(p + 1) if w0 is None else w0.copy()
    pen = np.full(p + 1, ridge); pen[0] = 0
    for _ in range(iters):
        eta = np.clip(Xb @ w, -30, 30); mu = 1 / (1 + np.exp(-eta))
        W = mu * (1 - mu)
        H = (Xb * W[:, None]).T @ Xb + np.diag(pen)
        step = np.linalg.solve(H, Xb.T @ (y - mu) - pen * w)
        w += step
        if np.max(np.abs(step)) < 1e-8:
            break
    eta = np.clip(Xb @ w, -30, 30)
    ll = float(np.sum(y * eta - np.log1p(np.exp(eta))))
    return w, ll
```

Code 17: Predicted probabilities (scripts/modeling.py)

```
def predict_logit(w, X):
    return 1 / (1 + np.exp(-np.clip(np.hstack([np.ones((X.shape[0], 1)), X]) @ w, -30, 30)))
```

7.2 Choosing splines by BIC

In each training fold, the full model is first fitted with straight lines; its BIC is $-2\ell + k \log n$ with k the number of columns plus the intercept. Then each candidate (`f_age`, `m_age`, `abs_age_diff`, `int_corr`, `fit_to_f_prefs`, `fit_to_m_prefs`) in turn gets a natural spline; a candidate whose spline lowers BIC is kept. The final design has all kept splines at once. Across the 360 fits in the grid, a spline was chosen 16 times, for `m_age`, so straight lines are adequate.

Code 18: Choosing splines by BIC (scripts/part1_models.py)

```
def choose_splines(Xtr, ytr, idtr, sets):
    """A candidate gets a natural cubic spline (3 df) when that lowers the full model's BIC on the training fold
    ."""
    base = make_prep(sets, Xtr).fit(Xtr, idtr); D = base.transform(Xtr).to_numpy()
    _, ll = M.fit_logit(D, ytr); b0 = M.bic(ll, D.shape[1] + 1, len(ytr))
    chosen = []
    for c in [c for c in sets["spline_candidates"] if c in Xtr.columns]:
        p = make_prep(sets, Xtr, splines=(c,)).fit(Xtr, idtr)
        if c not in p.knots:
            continue
        Dc = p.transform(Xtr).to_numpy(); _, llc = M.fit_logit(Dc, ytr)
        if M.bic(llc, Dc.shape[1] + 1, len(ytr)) < b0:
            chosen.append(c)
    return tuple(chosen)
```

Code 19: BIC (scripts/modeling.py)

```
def bic(ll, k, n):
    return -2 * ll + k * np.log(n)
```

7.3 Stepwise selection by BIC

Selection works on *blocks*: each numeric predictor with its spline columns, each binary column, and each categorical variable with all its dummies. **Forward** starts from the intercept and at each step

adds the block that lowers BIC most; **backward** starts from all blocks and removes the one whose removal lowers BIC most; **both ways** starts from the intercept and considers every addition and every removal at each step. Each stops when no single move lowers BIC. Each candidate model is a logistic regression fitted as above; the final model is refitted on the training fold and predicts the held-out pairs. Stepwise runs only under the headline and complete-case specifications (it is the slowest model).

Code 20: Stepwise selection over blocks (scripts/modeling.py)

```
def stepwise(X, y, blocks, direction):
    """Forward (from the intercept), backward (from all blocks) or both-ways (from the intercept) selection over
    blocks of columns, stopping when no single addition or removal lowers BIC. Returns the selected block names.
     """
    n = len(y)
    names = list(blocks)
    def score(sel):
        cols = [j for b in sel for j in blocks[b]]
        _, ll = fit_logit(X[:, cols], y) if cols else fit_logit(np.empty((n, 0)), y)
        return bic(ll, len(cols) + 1, n)
    sel = list(names) if direction == "backward" else []
    best = score(sel)
    while True:
        moves = []
        if direction in ("forward", "both"):
            moves += [("add", b) for b in names if b not in sel]
        if direction in ("backward", "both"):
            moves += [("drop", b) for b in sel]
        if not moves:
            return sel
        scored = [(score(sel + [b] if m == "add" else [s for s in sel if s != b]), m, b) for m, b in moves]
        s, m, b = min(scored, key=lambda t: t[0])
        if s >= best - 1e-9:
            return sel
        best = s
        sel = sel + [b] if m == "add" else [x for x in sel if x != b]
```

Code 21: The blocks stepwise selects (scripts/modeling.py)

```
    def blocks(self):
        """Stepwise blocks: each numeric predictor with its spline columns, each binary column, each categorical
     ."""
        out = {}
        for j, c in enumerate(self.columns):
            if c.startswith("cat:"):
                key = c[4:].split("=")[0]
            elif c.startswith("bin:"):
                key = c[4:]
            else:
                key = c.split(":ns")[0]
            out.setdefault(key, []).append(j)
        return out
```

7.4 Linear discriminant analysis (LDA)

LDA models the predictors within each class (match, no match) as multivariate normal with class means μ_1, μ_0 and a *shared* covariance matrix Σ, estimated from the training fold; the class priors π_1, π_0 are the training class frequencies. By Bayes' rule

$$P(\text{match} \mid x) = \frac{\pi_1 \phi(x; \mu_1, \Sigma)}{\pi_1 \phi(x; \mu_1, \Sigma) + \pi_0 \phi(x; \mu_0, \Sigma)},$$

which is a logistic function of a linear score, so LDA is a linear classifier fitted differently from logistic regression. We use scikit-learn's `LinearDiscriminantAnalysis` with its defaults (singular

value decomposition solver); there is nothing to tune.

7.5 Quadratic discriminant analysis (QDA)

QDA lets each class have its own covariance matrix Σ_k, so the boundary is quadratic. With more than 100 columns and 550 or so matches per training fold, the match class's covariance is poorly estimated, so it is regularized towards the identity: scikit-learn's `reg_param` r replaces each class covariance (in its scaled form) by $(1-r)\Sigma_k + rI$. r is tuned over $\{0.1, 0.3, 0.5, 0.7, 0.9\}$ by the inner folds (Section 5). The strongest smoothing (0.9) was chosen in 233 of 240 fits, typical of a weak signal.

Code 22: QDA (scripts/part1_models.py)

```
def make_qda(r):
    return QuadraticDiscriminantAnalysis(reg_param=r)
```

7.6 Random forest

An average of 500 classification trees, each grown on a bootstrap sample of the training fold; at each split a random $\sqrt{p}$ of the p columns are candidates (Gini impurity). The predicted probability is the average over trees of the share of matches in the leaf the pair falls into. The minimum leaf size is tuned over $\{1, 5, 20, 50\}$ by the inner folds. Seeds: `random_state` = 20261106 + fold. Predictions are computed on one thread so they reproduce bit for bit.

Code 23: Random forest (scripts/part1_models.py)

```
def make_rf(k, seed=RF_SEED):
    return lambda leaf: RandomForestClassifier(n_estimators=RF_TREES, max_features="sqrt", min_samples_leaf=leaf
    ,
                                               random_state=seed + k, n_jobs=RF_JOBS)
```

Code 24: Predictions on one thread (scripts/part1_models.py)

```
def predict_serial(model, D):
    """Predicted probabilities with one thread: a forest summing its trees across threads adds them in a varying
    order, so the last digits (and a tuning tie) could change from run to run."""
    if "n_jobs" in model.get_params():
        model.set_params(n_jobs=1)
    return model.predict_proba(D)[:, 1]
```

7.7 Lasso (L1-penalized logistic regression)

The lasso minimizes $\|\beta\|_1 + C\sum_i \text{log-loss}_i$ (scikit-learn `LogisticRegression` with `l1_ratio=1`, solver `liblinear`, `intercept_scaling=100` so the intercept is effectively unpenalized). Small C sets many coefficients exactly to zero. The grid has 30 values of C, log-spaced from $C_{\min} = 1/\max_j |\sum_i D_{ij}(y_i - \bar{y})|$ (the smallest penalty that zeroes every coefficient) to $1000\,C_{\min}$, tuned by the inner folds. The lasso was added after P1 and P2 had been tested, so it is reported but kept out of the "best model" they compare. Seeds: `random_state` = 20261206 + fold.

Code 25: The lasso penalty grid (scripts/part1_models.py)

```
def lasso_grid(D, y, steps=LASSO_STEPS):
    """Inverse penalties C for liblinear's L1 logistic regression, strongest penalty first: from the smallest
    penalty that zeroes every coefficient (C = 1 / max |D'(y - mean y)|) to one thousandth of it."""
    cmin = 1 / np.max(np.abs(D.T @ (y - y.mean())))
    return list(cmin * np.logspace(0, 3, steps))
```

Code 26: Lasso (scripts/part1_models.py)

```
def make_lasso(k):
    return lambda C: LogisticRegression(l1_ratio=1, solver="liblinear", C=C, intercept_scaling=100, tol=1e-4,
                                        random_state=LASSO_SEED + k)
```

7.8 The yes model (P5)

One row per date from one side, separately for women and men, on dates where the rater gave all six attribute ratings. Each rating is standardized within the rater's gender, and a logistic regression of the rater's yes on the six ratings is fitted on all those dates (no cross-validation; the coefficients are the result). Revealed importance of attribute a is its share of the positive standardized coefficients,

$$\text{revealed}_a = 100 \cdot \frac{\max(\hat{\beta}_a, 0)}{\sum_b \max(\hat{\beta}_b, 0)},$$

and stated importance is the mean own-preference share (`<a>1_1_share`) over participants of that gender in the 100-point waves who gave all six shares. The gap is revealed minus stated. Waves 6–9's stated shares are reported separately, without a verdict. Error bars: the cluster sandwich below, with events as clusters.

Code 27: The yes-model design (scripts/part1_models.py)

```
def yes_design(clean, g, exclude_events=()):
    """One gender's dates with all six ratings: (standardized ratings, yes, event)."""
    d = clean[(clean.gender == g) & clean[ATTRS].notna().all(axis=1) & ~clean.event.isin(exclude_events)]
    return (d[ATTRS] - d[ATTRS].mean()) / d[ATTRS].std(), d.dec.to_numpy(float), d.event.to_numpy(), d
```

Code 28: The yes model and stated-versus-revealed table (scripts/part1_models.py)

```
def yes_model(exclude_events=()):
    """Logistic regression of each date's yes on the six in-date ratings (standardized within the rater's gender
    ),
    by gender, on dates with all six ratings; cluster-robust sandwich by event with G/(G-1) (coefficient_table).
    Revealed importance = share of the positive standardized coefficients; stated importance = mean own-
     preference
    share in the 100-point waves (PREREG.md), with the waves 6 to 9 shares (rated 1 to 10, converted to shares)
    reported separately next to it. exclude_events leaves waves out of both (the wave 12 check)."""
    clean = pd.read_csv("data/clean.csv", low_memory=False)
    rows = []
    shares = [f"{a}1_1_share" for a in ATTRS]
    for g, name in ((0, "women"), (1, "men")):
        Z, yv, ev, d = yes_design(clean, g, exclude_events)
        tab, _ = coefficient_table(Z, yv, ev)
        coef, se = tab.set_index("term").coef[ATTRS], tab.set_index("term").se_cluster_event[ATTRS]
        pos = coef.clip(lower=0)
        people = clean[(clean.gender == g) & ~clean.event.isin(exclude_events)].drop_duplicates("rater")
        p100 = people[(people.pref_scale == "100pt") & people[shares].notna().all(axis=1)]
        p69 = people[(people.pref_scale == "1-10") & people[shares].notna().all(axis=1)]
        stated, stated69 = p100[shares].mean(), p69[shares].mean()
        for a in ATTRS:
            rows.append(dict(rater_gender=name, attribute=a, std_coef=coef[a], se_cluster_event=se[a],
                             revealed_share=100 * pos[a] / pos.sum(), stated_share=stated[f"{a}1_1_share"],
                             stated_share_waves_6_9=stated69[f"{a}1_1_share"], dates=len(d), raters=d.rater.
     nunique(),
                             stated_people=len(p100), stated_people_waves_6_9=len(p69),
                             excluded_events=";".join(map(str, exclude_events)), fold_scheme="none (all dates)")
    )
    t = pd.DataFrame(rows)
    t["gap_revealed_minus_stated"] = t.revealed_share - t.stated_share
    t["gap_revealed_minus_stated_waves_6_9"] = t.revealed_share - t.stated_share_waves_6_9
    return t
```

7.9 The liking model (L1, L2)

One row per date from one side, on dates with a liking rating (1–10). Predictors: the rater's and the partner's sign-up answers (the `pre_individual` columns relabelled rater `r_` and partner `p_`), the rater's gender, and the eleven pair features of P3b, with the compatibility terms relabelled as fit to the rater's and to the partner's priorities. Preparation is the same as for the match models (no splines). The model is ordinary least squares, fitted in each training fold under the same event folds. Ladder: a baseline that predicts each held-out date by its training fold's mean liking; rater only; plus partner; plus pair features. A complete-case version uses only dates complete on every predictor.

Code 29: One row per date from one side (scripts/like_models.py)

```
def dates_table(pairs, sets, clean):
    """One row per date from one side: the rater's answers as r_*, the partner's as p_*, the pair features, the
    rater's gender (r_male) and the like rating. Built from the headline match design, so every preparation
    decision that comes before the training folds (format standardization, recording-error fix) is shared."""
    X, _, ev, ids = M.build_frame(pairs, sets, M.HEADLINE, "pre_extended")
    X = X.assign(event=ev, f_iid=ids.f_iid, m_iid=ids.m_iid)
    person = [c[2:] for c in X.columns if c.startswith("f_") and c != "f_iid"]       # stems shared by f_ and m_
    pair = [{"fit_to_f_prefs": "fit_to_rater_prefs", "fit_to_m_prefs": "fit_to_partner_prefs"}.get(c, c)
            for c in X.columns if not c.startswith(("f_", "m_")) and c != "event"]          # relabelled from the
      rater's side
    rows = []
    for g, me, other in ((0, "f", "m"), (1, "m", "f")):
        part = X.rename(columns={**{f"{me}_{s}": f"r_{s}" for s in person}, **{f"{other}_{s}": f"p_{s}" for s in
     person},
                                 f"{me}_iid": "r_iid", f"{other}_iid": "p_iid",
                                 f"fit_to_{me}_prefs": "fit_to_rater_prefs", f"fit_to_{other}_prefs": "
    fit_to_partner_prefs"})
        rows.append(part.assign(r_male=g))
    D = pd.concat(rows, ignore_index=True)
    like = clean.set_index(["rater", "ratee"]).like
    D["like"] = like.reindex(pd.MultiIndex.from_arrays([D.r_iid, D.p_iid])).to_numpy()
    D = D[D.like.notna()].reset_index(drop=True)
    rater = [f"r_{s}" for s in person] + ["r_male"]
    partner = [f"p_{s}" for s in person]
    return D, rater, partner, pair
```

Code 30: The liking model in one fold (scripts/like_models.py)

```
def fit_fold(k, D, cols, folds, sets):
    test = D.event.map(folds).to_numpy() == k; train = ~test
    if not cols:                                                    # the baseline: the training fold's mean
        return np.where(test)[0], np.full(test.sum(), D.like.to_numpy()[train].mean())
    Xtr, Xte = D.loc[train, cols].reset_index(drop=True), D.loc[test, cols].reset_index(drop=True)
    ids = D.loc[train, ["r_iid", "p_iid"]].reset_index(drop=True)
    cat, binary = column_types(cols, sets)
    person = [c for c in cols if c.startswith(("r_", "p_")) and c not in cat + binary]
    p = M.Prep(categorical=cat, binary=binary, person_cols=person, sides=("r", "p")).fit(Xtr, ids)
    A, B = p.transform(Xtr).to_numpy(), p.transform(Xte).to_numpy()
    A1, B1 = np.column_stack([np.ones(len(A)), A]), np.column_stack([np.ones(len(B)), B])
    beta = np.linalg.lstsq(A1, D.like.to_numpy()[train], rcond=None)[0]
    return np.where(test)[0], B1 @ beta
```

8 Metrics and error bars

8.1 The metrics

With $\hat{y}_i = 1$ when $\hat{p}_i$ exceeds its fold's threshold, $c_i = 1$ when $\hat{y}_i = y_i$, n pairs, n_1 matches and n_0 non-matches:

$$\text{error rate} = \frac{1}{n}\sum_i (1 - c_i), \qquad \text{sensitivity} = \frac{1}{n_1}\sum_{i:y_i=1} c_i,$$

$$\text{specificity} = \frac{1}{n_0}\sum_{i:y_i=0} c_i, \qquad \text{balanced accuracy} = \tfrac{1}{2}(\text{sensitivity} + \text{specificity}).$$

Since 83.5% of pairs do not match, the error rate alone rewards predicting "no" for everyone; balanced accuracy and AUC do not.

AUC is the probability that a randomly chosen match gets a higher predicted probability than a randomly chosen non-match (ties count half), computed from ranks (the Mann–Whitney statistic): with R_i the rank of $\hat{p}_i$ among all n pairs,

$$\text{AUC} = \frac{\sum_{i:y_i=1} R_i - n_1(n_1+1)/2}{n_1 n_0}.$$

0.5 is chance and 1 is perfect. It uses the predicted probabilities directly, so it needs no threshold.

Code 31: AUC from ranks (scripts/modeling.py)

```python
def auc(score, y):
    """Probability a random match outranks a random non-match (ties count half)."""
    score, y = np.asarray(score, float), np.asarray(y)
    r = pd.Series(score).rank().to_numpy()
    n1 = (y == 1).sum(); n0 = len(y) - n1
    return (r[y == 1].sum() - n1 * (n1 + 1) / 2) / (n1 * n0)
```

Code 32: The metric table with its error bars (scripts/modeling.py)

```python
def metric_table(prob, y, threshold, g):
    """Event-held-out metrics with cluster-robust standard errors (clusters g = events). prob and threshold are
    per dyad (the threshold of the fold that scored it). Each metric is written as a mean of per-dyad
    contributions whose residuals are the linearized ones, so the cluster-robust formula applies: sensitivity is
    a ratio (correct matches over matches), so a matched dyad's residual is (correct - sensitivity) * n / n1 and
    an unmatched dyad's is 0; specificity likewise; balanced accuracy averages the two."""
    prob, y, threshold, g = map(np.asarray, (prob, y, threshold, g))
    pred = (prob > threshold).astype(float)
    n, n1 = len(y), (y == 1).sum(); n0 = n - n1
    correct = (pred == y).astype(float)
    sens, spec = correct[y == 1].mean(), correct[y == 0].mean()
    r_sens = np.where(y == 1, (correct - sens) * n / n1, 0.0)
    r_spec = np.where(y == 0, (correct - spec) * n / n0, 0.0)
    contrib = {
        "error_rate": 1 - correct,
        "sensitivity": sens + r_sens,
        "specificity": spec + r_spec,
        "balanced_accuracy": (sens + spec) / 2 + (r_sens + r_spec) / 2,
    }
    out = {k: (v.mean(), np.sqrt(cluster_robust_var_of_mean(v, g))) for k, v in contrib.items()}
    out["auc"] = (auc(prob, y), np.sqrt(auc_clustered_var(prob, y, g)))
    return out
```

8.2 Cluster-robust error bars

Pairs are not independent: the same people appear in many pairs, and pairs at an event share an evening. All error bars treat the $G = 21$ events as clusters. A metric written as a mean of per-pair contributions u_i has variance

$$\widehat{\mathrm{Var}}(\bar{u}) = \frac{G}{G-1} \cdot \frac{1}{n^2} \sum_{g=1}^{G} \Big(\sum_{i \in g} (u_i - \bar{u}) \Big)^2$$

(convention 6; `modeling.cluster_robust_var_of_mean`). The error rate is a plain mean ($u_i = 1 - c_i$). Sensitivity, specificity and balanced accuracy are ratios, so they use linearized contributions: for sensitivity a matched pair contributes $\widehat{\mathrm{Se}} + (c_i - \widehat{\mathrm{Se}})\, n/n_1$ and an unmatched pair $\widehat{\mathrm{Se}}$; specificity likewise with n_0; balanced accuracy averages the two residuals.

AUC uses Obuchowski's (1997) clustered variance. Each match i gets the placement value $V_{10,i}$, the share of non-matches it outranks, and each non-match j gets $V_{01,j}$, the share of matches that outrank it (ties half); the AUC A is the mean of either. With cluster sums $a_g = \sum_{i \in g, y_i = 1} (V_{10,i} - A)$ and $b_g = \sum_{j \in g, y_j = 0} (V_{01,j} - A)$,

$$\widehat{\mathrm{Var}}(\mathrm{AUC}) = \frac{G}{G-1} \sum_{g} \Big(\frac{a_g}{n_1} + \frac{b_g}{n_0} \Big)^2 .$$

Liking R-squared $= 1 - A/B$, with A the mean squared error and B the mean squared deviation of liking from its mean, uses the linearized contribution $-(e_i - A)/B + A(d_i - B)/B^2$ (e_i the squared error, d_i the squared deviation); RMSE $= \sqrt{A}$ uses the delta method.

Coefficients (yes model and coefficient table) use the cluster sandwich $\hat{V} = B \left(\frac{G}{G-1} \sum_g s_g s_g^\top \right) B$ with $B = (X^\top W X + \Lambda)^{-1}$ and s_g the sum of the score contributions $x_i(y_i - \hat{p}_i)$ over cluster g.

Every one of these error bars was checked by simulation before being reported (Section 12).

Code 33: Cluster-robust variance of a mean (convention 6) (scripts/modeling.py)

```
def cluster_robust_var_of_mean(u, g):
    """Variance of mean(u) with clusters g: G/(G-1) * sum over clusters of (within-cluster sum of residuals)^2 /
      n^2
    (the unbalanced form in convention 6)."""
    u = np.asarray(u, float); n = len(u); res = u - u.mean()
    tot = pd.Series(res).groupby(np.asarray(g)).sum().to_numpy()
    G = len(tot)
    return G / (G - 1) * np.sum(tot ** 2) / n ** 2
```

Code 34: Placement values (scripts/modeling.py)

```
def placement_values(score, y):
    """Per-positive V10 (share of negatives scored below it, ties half) and per-negative V01 (share of positives
    scored above it)."""
    score, y = np.asarray(score, float), np.asarray(y)
    pos, neg = np.sort(score[y == 1]), np.sort(score[y == 0])
    s1, s0 = score[y == 1], score[y == 0]
    v10 = (np.searchsorted(neg, s1, "left") + np.searchsorted(neg, s1, "right")) / 2 / len(neg)
    v01 = 1 - (np.searchsorted(pos, s0, "left") + np.searchsorted(pos, s0, "right")) / 2 / len(pos)
    return v10, v01
```

Code 35: Clustered AUC variance (Obuchowski 1997) (scripts/modeling.py)

```
def auc_clustered_var(score, y, g):
    """Obuchowski (1997) variance of the AUC when observations come in clusters g."""
    score, y, g = np.asarray(score, float), np.asarray(y), np.asarray(g)
    v10, v01 = placement_values(score, y)
```

```
    A = v10.mean(); M, N = len(v10), len(v01)
    cl = np.unique(g)
    m_k = pd.Series(y == 1).groupby(g).sum().reindex(cl, fill_value=0).to_numpy(float)
    n_k = pd.Series(y == 0).groupby(g).sum().reindex(cl, fill_value=0).to_numpy(float)
    V10 = pd.Series(v10).groupby(g[y == 1]).sum().reindex(cl, fill_value=0).to_numpy()
    V01 = pd.Series(v01).groupby(g[y == 0]).sum().reindex(cl, fill_value=0).to_numpy()
    K = len(cl)
    a, b = V10 - m_k * A, V01 - n_k * A
    S10 = K / ((K - 1) * M) * np.sum(a ** 2)
    S01 = K / ((K - 1) * N) * np.sum(b ** 2)
    S11 = K / (K - 1) * np.sum(a * b)
    return S10 / M + S01 / N + 2 * S11 / (M * N)
```

Code 36: Liking R-squared and RMSE with their error bars (scripts/modeling.py)

```
def r_squared_and_rmse(y, pred, g):
    """Held-out R-squared and RMSE with cluster-robust SEs (clusters g = events). R-squared = 1 - A / B with A
     the
    mean squared error and B the mean squared deviation of y from its mean; each date contributes the linearized
    residual -(e_i - A) / B + A (d_i - B) / B^2, with e_i its squared error and d_i its squared deviation, to
     the
    cluster-robust variance of a mean (convention 6). RMSE = sqrt(A), SE by the delta method."""
    y, pred, g = np.asarray(y, float), np.asarray(pred, float), np.asarray(g)
    e, d = (y - pred) ** 2, (y - y.mean()) ** 2
    A, B = e.mean(), d.mean()
    z = -(e - A) / B + A * (d - B) / B ** 2
    se_a = np.sqrt(cluster_robust_var_of_mean(e, g))
    return dict(r_squared=1 - A / B, r_squared_se=np.sqrt(cluster_robust_var_of_mean(1 - A / B + z, g)),
                rmse=np.sqrt(A), rmse_se=se_a / (2 * np.sqrt(A)))
```

9 How each hypothesis is decided

The "best model" for P1 and P2 is the highest AUC among the pre-registered pool: full logistic regression, the three stepwise directions, LDA, QDA and the random forest (the lasso is excluded, and stepwise is not run in the wave checks). All AUCs come from event cross-validation under the stated specification.

	Statistic	**Supported when**
P1	Best AUC on `pre`	between 0.58 and 0.70
P2	Best AUC on `pre+date` minus best on `pre`	at least 0.10
P3	Full logistic AUC on `pre` minus on `pre_individual`	less than 0.02
P3b	Full logistic AUC on `pre_extended` minus on `pre`	less than 0.02
P3c	Full logistic AUC on `pre_interactions` minus on `pre_extended`	less than 0.02
P4	Random forest AUC on `pre` minus full logistic on `pre`	less than 0.03
P5	Attractiveness has the largest standardized coefficient and the largest gap (revealed minus stated), for each gender	both true
L1	Held-out R-squared of the full liking model	less than 0.10
L2	Full liking model minus rater + partner, in held-out R-squared	less than 0.01

Current results (read from the hypothesis tables):

	Version	Statistic	Verdict
P1	headline	best pre AUC 0.5825 (rf, SE 0.0197)	supported
P2	headline	best pre+date AUC 0.8459 (rf) minus best pre 0.5825 = 0.2634	supported
P3	headline	full logistic pre AUC 0.5425 minus without pair features 0.5319 = 0.0105	supported
P3b	headline	full logistic AUC with extended pair features 0.5454 minus with the six pair features 0.5425 = 0.0030	supported
P3c	headline	plus the three declared terms 0.5465 minus 0.5454 = 0.0010	supported
P4	headline	random forest pre AUC 0.5825 minus full logistic 0.5425 = 0.0401	not supported
P5	women	largest std. coefficient: attr; largest revealed-minus-stated gap: attr (+18.4 points)	supported
P5	men	largest std. coefficient: attr; largest revealed-minus-stated gap: attr (+25.3 points)	supported
L1	headline	held-out R-squared of the full model -0.0341 (SE 0.0304)	supported
L2	headline	pair features add 0.0084 (-0.0341 minus -0.0425)	supported

10 Robustness and sensitivity checks

The 24-specification grid. Every combination of these choices ($3 \times 2 \times 2 \times 2$) is run for every model (stepwise only under the headline and complete-case ones) and reported in one table; the headline was fixed in `PREREG.md` before any model ran.

Decision	Choices
Missing values	median fill plus all flags (headline); median fill with sign-up flags only; complete cases (dyads complete on every `pre+date` column, no filling, no flags)
Preference fit	compatibility term B (headline); plain weighted mean A
Interest values	gaming 14 and reading 13 missing (headline); as recorded, with the released `int_corr`
Survey format	shares standardized within format (headline); unstandardized

Complete cases. Every results table has a complete-case version: only the dyads with every column of the feature set recorded (2,685 dyads, 477 matches), nothing filled, no flags. The pair-feature ladder's complete-case version uses the dyads complete on every column any step uses, so all steps share the same dyads.

Wave checks. P1 to P4 are recomputed without waves 6–9 (the 1–10 survey format) and without wave 12 (yes limited to half one's dates), every model except stepwise, in three versions: the remaining events dealt into 5 new folds by the same seed (the pre-registered version), the headline folds minus the dropped waves, and complete cases. P5 is also recomputed without wave 12.

Fold and seed noise (exploratory). The headline full logistic model is rerun with 10 other fold seeds and the headline forest with 5 other forest seeds, to show how much the AUCs move for reasons unrelated to the data.

11 Diagnostics

- **Coefficient table** (full logistic model on all dyads, headline and complete-case). Identical columns are reported as one combined term. A coefficient that moves by more than 0.01 when the ridge goes from 10^{-4} to 10^{-8} has no finite estimate (separation: some combination of columns

occurs only with non-matches) and is reported as not estimable, with the sum of the separated terms where that sum is stable. SEs are the cluster sandwich; a bootstrap shows they are about 20% too narrow on this design, so read the z-values with care.

- **Variance inflation factors**: $1/(1 - R_j^2)$ from regressing each column on all the others; an exactly reproduced column is reported as infinite.
- **Calibration**: held-out predictions in ten equal-sized groups by predicted probability, mean prediction against observed match rate.
- **Binned residuals**: for each continuous predictor, held-out residuals (observed minus predicted) averaged in ten bins, with ±2 SE bands, to spot curvature.
- **Cook's distance and leverage** of the full-data fit, the ten most influential dyads listed.

12 Simulation checks of the error bars

Each error-bar formula was checked on simulated data with the real design before being reported (convention 7). The ratio of the average SE to the true spread of the estimate should be near 1, and 90% intervals should cover the true value about 90% of the time.

Error bar	SE / true spread	90% coverage
AUC (match models)	0.84 to 0.95	80% to 87%
Sensitivity, specificity, balanced accuracy	0.89 to 0.96	82% to 87%
Liking R-squared	0.97 to 1.07	87% to 90%
Liking RMSE	0.94 to 1.01	85% to 88%
Yes-model coefficients	0.90 to 1.15	85% to 90%
Coefficient table	median 0.79	median 80%

Designs: for the match metrics, the 21 real event sizes with an event effect on the match probability (SD 0.6, and 0.4 to match the real spread of match rates across events) and a score correlated within events, five seeds each (`sim_auc_cluster.py`); for liking, the real event sizes with event, rater and partner effects (`sim_like_cluster.py`); for coefficients, a parametric bootstrap on the real designs with an event effect of SD 0.4 (`sim_coef_cluster.py`). With 21 events the match-model error bars run slightly narrow: read a 90% interval as roughly an 83% one.

13 Reproducing the results

1. Create the environment: `python3 -m venv .venv`, then `.venv/bin/pip install -r requirements.txt`.
2. Run the scripts in the order of Section 1 from the repository root. `part1_models.py` takes about 30 minutes with `--jobs 5`; set `OMP_NUM_THREADS=1` so parallel folds do not compete for threads.
3. Run the tests: `for t in tests/test_*.py; do .venv/bin/python $t; done`.
4. Regenerate this document: `python scripts/methods_doc.py`.

The same code and seeds reproduce every number to the last digit (checked by two independent reviews).

A Every predictor

Every column of `pre` and `pre+date`, as listed in `data/feature_sets.json`, and the four missing-answer flags, with the number of dyads where each is missing before filling (of 4,184). Columns marked * are left out of every model (redundant shares).

Column	Meaning	Scale	Missing
`f_age`	Woman's answer: Age in years	years	65
`f_field_cd`	Woman's answer: Field of study, coded: 1 Law, 2 Math, 3 Social science/psychologist, 4 Medical science/pharma/biotech, 5 Engineering, 6 English/creative writing/journalism, 7 History/religion/philosophy, 8 Business/econ/finance, 9 Education/academia, 10 Biological sciences/chemistry/physics, 11 Social work, 12 Undergrad/undecided, 13 Political science/international affairs, 14 Film, 15 Fine arts/arts administration, 16 Languages, 17 Architecture, 18 Other	code (categorical)	43
`f_career_c`	Woman's answer: Intended career, coded: 1 Lawyer, 2 Academic/research, 3 Psychologist, 4 Doctor/medicine, 5 Engineer, 6 Creative arts/entertainment, 7 Banking/consulting/finance/marketing/business/CEO/entrepreneur/admin, 8 Real estate, 9 International/humanitarian affairs, 10 Undecided, 11 Social work, 12 Speech pathology, 13 Politics, 14 Pro sports/athletics, 15 Other, 16 Journalism, 17 Architecture	code (categorical)	99
`f_goal`	Woman's answer: Primary goal in coming: 1 Seemed like a fun night out, 2 To meet new people, 3 To get a date, 4 Looking for a serious relationship, 5 To say I did it, 6 Other	code (categorical)	59
`f_date`	Woman's answer: How often do you go on dates? 1 Several times a week, 2 Twice a week, 3 Once a week, 4 Twice a month, 5 Once a month, 6 Several times a year, 7 Almost never	1 to 7 (1 = most often)	59
`f_go_out`	Woman's answer: How often do you go out (not necessarily on dates)? Same scale as dating frequency	1 to 7 (1 = most often)	59
`f_exphappy`	Woman's answer: How happy do you expect to be with the people you meet at the event?	1 to 10	81
`f_sports`	Woman's interest in playing sports/athletics	1 to 10	59
`f_tvsports`	Woman's interest in watching sports	1 to 10	59
`f_exercise`	Woman's interest in body building/exercising	1 to 10	59
`f_dining`	Woman's interest in dining out	1 to 10	59
`f_museums`	Woman's interest in museums/galleries	1 to 10	59
`f_art`	Woman's interest in art	1 to 10	59
`f_hiking`	Woman's interest in hiking/camping	1 to 10	59
`f_gaming`	Woman's interest in gaming	1 to 10	84
`f_clubbing`	Woman's interest in dancing/clubbing	1 to 10	59
`f_reading`	Woman's interest in reading	1 to 10	81
`f_tv`	Woman's interest in watching tv	1 to 10	59
`f_theater`	Woman's interest in theater	1 to 10	59
`f_movies`	Woman's interest in movies	1 to 10	59
`f_concerts`	Woman's interest in going to concerts	1 to 10	59
`f_music`	Woman's interest in music	1 to 10	59
`f_shopping`	Woman's interest in shopping	1 to 10	59
`f_yoga`	Woman's interest in yoga/meditation	1 to 10	59
`f_self_attr`	Woman's self-rating: how attractive am I?	1 to 10	67
`f_self_sinc`	Woman's self-rating: how sincere am I?	1 to 10	67
`f_self_intel`	Woman's self-rating: how intelligent am I?	1 to 10	67

Column	Meaning	Scale	Missing
f_self_fun	Woman's self-rating: how fun am I?	1 to 10	67
f_self_amb	Woman's self-rating: how ambitious am I?	1 to 10	67
f_pref_attr	Woman's stated importance of attractive in a date, as a share of 100	share (sums to 100)	59
f_pref_sinc	Woman's stated importance of sincere in a date, as a share of 100	share (sums to 100)	59
f_pref_intel	Woman's stated importance of intelligent in a date, as a share of 100	share (sums to 100)	59
f_pref_fun	Woman's stated importance of fun in a date, as a share of 100	share (sums to 100)	59
f_pref_amb	Woman's stated importance of ambitious in a date, as a share of 100	share (sums to 100)	59
f_pref_shar*	Woman's stated importance of shared interests/hobbies in a date, as a share of 100	share (sums to 100)	59
m_age	Man's answer: Age in years	years	29
m_field_cd	Man's answer: Field of study, coded: 1 Law, 2 Math, 3 Social science/psychologist, 4 Medical science/pharma/biotech, 5 Engineering, 6 English/creative writing/journalism, 7 History/religion/philosophy, 8 Business/econ/finance, 9 Education/academia, 10 Biological sciences/chemistry/physics, 11 Social work, 12 Undergrad/undecided, 13 Political science/international affairs, 14 Film, 15 Fine arts/arts administration, 16 Languages, 17 Architecture, 18 Other	code (categorical)	39
m_career_c	Man's answer: Intended career, coded: 1 Lawyer, 2 Academic/research, 3 Psychologist, 4 Doctor/medicine, 5 Engineer, 6 Creative arts/entertainment, 7 Banking/consulting/finance/marketing/business/CEO/entrepreneur/admin, 8 Real estate, 9 International/humanitarian affairs, 10 Undecided, 11 Social work, 12 Speech pathology, 13 Politics, 14 Pro sports/athletics, 15 Other, 16 Journalism, 17 Architecture	code (categorical)	39
m_goal	Man's answer: Primary goal in coming: 1 Seemed like a fun night out, 2 To meet new people, 3 To get a date, 4 Looking for a serious relationship, 5 To say I did it, 6 Other	code (categorical)	20
m_date	Man's answer: How often do you go on dates? 1 Several times a week, 2 Twice a week, 3 Once a week, 4 Twice a month, 5 Once a month, 6 Several times a year, 7 Almost never	1 to 7 (1 = most often)	38
m_go_out	Man's answer: How often do you go out (not necessarily on dates)? Same scale as dating frequency	1 to 7 (1 = most often)	20
m_exphappy	Man's answer: How happy do you expect to be with the people you meet at the event?	1 to 10	20
m_sports	Man's interest in playing sports/athletics	1 to 10	20
m_tvsports	Man's interest in watching sports	1 to 10	20
m_exercise	Man's interest in body building/exercising	1 to 10	20
m_dining	Man's interest in dining out	1 to 10	20
m_museums	Man's interest in museums/galleries	1 to 10	20
m_art	Man's interest in art	1 to 10	20
m_hiking	Man's interest in hiking/camping	1 to 10	20
m_gaming	Man's interest in gaming	1 to 10	73
m_clubbing	Man's interest in dancing/clubbing	1 to 10	20
m_reading	Man's interest in reading	1 to 10	49
m_tv	Man's interest in watching tv	1 to 10	20
m_theater	Man's interest in theater	1 to 10	20
m_movies	Man's interest in movies	1 to 10	20

Column	Meaning	Scale	Missing
m_concerts	Man's interest in going to concerts	1 to 10	20
m_music	Man's interest in music	1 to 10	20
m_shopping	Man's interest in shopping	1 to 10	20
m_yoga	Man's interest in yoga/meditation	1 to 10	20
m_self_attr	Man's self-rating: how attractive am I?	1 to 10	38
m_self_sinc	Man's self-rating: how sincere am I?	1 to 10	38
m_self_intel	Man's self-rating: how intelligent am I?	1 to 10	38
m_self_fun	Man's self-rating: how fun am I?	1 to 10	38
m_self_amb	Man's self-rating: how ambitious am I?	1 to 10	38
m_pref_attr	Man's stated importance of attractive in a date, as a share of 100	share (sums to 100)	20
m_pref_sinc	Man's stated importance of sincere in a date, as a share of 100	share (sums to 100)	20
m_pref_intel	Man's stated importance of intelligent in a date, as a share of 100	share (sums to 100)	20
m_pref_fun	Man's stated importance of fun in a date, as a share of 100	share (sums to 100)	20
m_pref_amb	Man's stated importance of ambitious in a date, as a share of 100	share (sums to 100)	20
m_pref_shar*	Man's stated importance of shared interests/hobbies in a date, as a share of 100	share (sums to 100)	20
abs_age_diff	Age gap |man's age - woman's age|	years	94
int_corr	Interest similarity: Pearson correlation of the two people's ratings of the 16 interests other than yoga	-1 to 1	79
same_field	1 if the two field codes are equal	0/1	82
same_goal	1 if the two goal codes are equal	0/1	79
fit_to_f_prefs	How well the man's relative strengths fit the woman's priorities (compatibility term, equation 1)	standardized	97
fit_to_m_prefs	How well the woman's relative strengths fit the man's priorities	standardized	87
f_rating_attr	Woman's scorecard rating of the partner: how attractive is this person?	1 to 10	101
f_rating_sinc	Woman's scorecard rating of the partner: how sincere is this person?	1 to 10	130
f_rating_intel	Woman's scorecard rating of the partner: how intelligent is this person?	1 to 10	147
f_rating_fun	Woman's scorecard rating of the partner: how fun is this person?	1 to 10	183
f_rating_amb	Woman's scorecard rating of the partner: how ambitious is this person?	1 to 10	362
f_rating_shar	Woman's scorecard rating of the partner: how shared interests/hobbies is this person?	1 to 10	576
f_rating_like	Woman's scorecard rating of the partner: overall, how much do you like this person?	1 to 10	135
f_prob	Woman's guess: how probable is it that this person will say yes to you?	1 to 10	164
m_rating_attr	Man's scorecard rating of the partner: how attractive is this person?	1 to 10	101
m_rating_sinc	Man's scorecard rating of the partner: how sincere is this person?	1 to 10	147
m_rating_intel	Man's scorecard rating of the partner: how intelligent is this person?	1 to 10	149
m_rating_fun	Man's scorecard rating of the partner: how fun is this person?	1 to 10	167
m_rating_amb	Man's scorecard rating of the partner: how ambitious is this person?	1 to 10	350
m_rating_shar	Man's scorecard rating of the partner: how shared interests/hobbies is this person?	1 to 10	490

Column	Meaning	Scale	Missing
m_rating_like	Man's scorecard rating of the partner: overall, how much do you like this person?	1 to 10	105
m_prob	Man's guess: how probable is it that this person will say yes to you?	1 to 10	144
f_pre_missing	Woman's sign-up form has an answer left blank	0/1	0
m_pre_missing	Man's sign-up form has an answer left blank	0/1	0
f_date_missing	Woman's scorecard has a rating left blank	0/1	0
m_date_missing	Man's scorecard has a rating left blank	0/1	0

B Seeds

Random operation	Seed
Outer event folds	20261006
Inner tuning folds	20261006 + 1 + outer fold
Random forest	20261106 + outer fold
Lasso	20261206 + outer fold
AUC simulation	20261006 to 20261015 (five per design)
Liking simulation	20261506 to 20261515
Coefficient bootstrap	20261406 + 1000 x design + replicate
Fold and seed noise	fold seeds 20261007 to 20261016; forest seeds 20261107 to 20261111

C More code

The rest of the code that decides the reported results, as it ran.

Code 37: How P1 to P4 are decided (scripts/part1_models.py)

```
def hypotheses(met, sr):
    h = met[met.headline.astype(bool)]
    cand = ["logistic", "stepwise_forward", "stepwise_backward", "stepwise_both", "lda", "qda", "rf"]
    def best(df, fs):
        d = df[(df.feature_set == fs) & df.model.isin(cand)]
        return d.loc[d.auc.idxmax()]
    rows = []
    for label, df in (("headline", h), ("complete-case", met[met.spec == M.COMPLETE_CASE.label])):
        if df.empty or "pre" not in set(df.feature_set):
            continue
        bp, bd = best(df, "pre"), best(df, "pre+date")
        rows.append(dict(hypothesis="P1", version=label, statistic=f"best `pre` AUC {bp.auc:.4f} ({bp.model}, SE
     {bp.auc_se:.4f})",
                         supported=bool(0.58 <= bp.auc < 0.70), note=""))
        gap = bd.auc - bp.auc
        rows.append(dict(hypothesis="P2", version=label, statistic=f"best `pre+date` AUC {bd.auc:.4f} ({bd.model
    }) minus best `pre` {bp.auc:.4f} = {gap:.4f}",
                         supported=bool(gap >= 0.10), note=""))
        lp = df[(df.feature_set == "pre") & (df.model == "logistic")]; li = df[(df.feature_set == "
    pre_individual") & (df.model == "logistic")]
        if len(lp) and len(li):
            d3 = lp.auc.iloc[0] - li.auc.iloc[0]
            rows.append(dict(hypothesis="P3", version=label, statistic=f"full logistic `pre` AUC {lp.auc.iloc
    [0]:.4f} minus without pair features {li.auc.iloc[0]:.4f} = {d3:.4f}",
                             supported=bool(d3 < 0.02), note=""))
        rf = df[(df.feature_set == "pre") & (df.model == "rf")]
        if len(lp) and len(rf):
            d4 = rf.auc.iloc[0] - lp.auc.iloc[0]
```

```
            rows.append(dict(hypothesis="P4", version=label, statistic=f"random forest `pre` AUC {rf.auc.iloc
    [0]:.4f} minus full logistic {lp.auc.iloc[0]:.4f} = {d4:.4f}",
                             supported=bool(d4 < 0.03), note=""))
    for r in rows:
        r["fold_scheme"] = "event"
    rows += p5_rows(sr)
    if "gap_revealed_minus_stated_waves_6_9" in sr:
        rows += p5_rows(sr, "{g}, waves 6 to 9 stated shares", "gap_revealed_minus_stated_waves_6_9", verdict=
    False,
                        note="reported separately without a verdict (PREREG.md, survey format); same yes model")
    return pd.DataFrame(rows)
```

Code 38: How P5 is decided (scripts/part1_models.py)

```
def p5_rows(sr, version="{g}", col="gap_revealed_minus_stated", verdict=True,
            note="dates with all six ratings; stated shares from the 100-point waves"):
    """P5 as tested since its first run: attractiveness has the largest standardized coefficient and the largest
    signed gap (revealed minus stated). The margin over the next gap and the reading by absolute size are
    exploratory. verdict=False reports a version without a verdict (the waves 6 to 9 stated shares)."""
    rows = []
    for g in ("women", "men"):
        s = sr[sr.rater_gender == g].set_index("attribute")
        top_coef, gaps = s.std_coef.idxmax(), s[col].sort_values(ascending=False)
        size = s[col].abs().sort_values(ascending=False)
        rows.append(dict(hypothesis="P5", version=version.format(g=g), fold_scheme="none (all dates)",
                         statistic=f"largest std. coefficient: {top_coef}; largest revealed-minus-stated gap: {
    gaps.index[0]} ({gaps.iloc[0]:+.1f} points)",
                         supported=bool(top_coef == "attr" and gaps.index[0] == "attr") if verdict else None,
                         note=f"{note}; exploratory: the largest gap leads the next ({gaps.index[1]}) by {gaps.
    iloc[0] - gaps.iloc[1]:.2f} points, "
                              f"and by absolute size the largest is {size.index[0]}, ahead of {size.index[1]} by
     {size.iloc[0] - size.iloc[1]:.2f} points"))
    return rows
```

Code 39: Identical columns (scripts/part1_models.py)

```
def twin_groups(D):
    """Groups of columns that are identical over every row of D, in column order."""
    seen = {}
    for c in D.columns:
        seen.setdefault(D[c].to_numpy(float).tobytes(), []).append(c)
    return [grp for grp in seen.values() if len(grp) > 1]
```

Code 40: The coefficient table (scripts/part1_models.py)

```
def coefficient_table(D, y, g, ridge=1e-6, path=(1e-4, 1e-8), tol=0.01):
    """Logistic regression by IRLS (modeling.fit_logit, run to convergence) with the cluster-robust sandwich:
     bread
    (X'WX + ridge)^-1, meat G/(G-1) times the sum over clusters of the outer products of the score sums.
    - Identical columns: only their combined effect is estimable, so each group is fitted as one column and
     reported
      as one term, its names joined by ' + '.
    - Separation: a coefficient that moves by more than tol when the ridge goes from path[0] to path[1] has no
      finite estimate (the likelihood keeps rising along some direction, through one column or several together)
     ,
      so it is reported as not estimable; where several move and their sum does not, the sum gets its own row.
     The
      other coefficients and SEs are the limit of the likelihood, since the separated dyads' fitted
     probabilities
      sit at 0 or 1 and add nothing.
    Returns (table, fitted probabilities)."""
    y, g = np.asarray(y, float), np.asarray(g); n = len(y)
    groups = twin_groups(D)
    R = D.drop(columns=[c for grp in groups for c in grp[1:]])
    names = {grp[0]: " + ".join(grp) for grp in groups}
    terms = np.array(["const"] + [names.get(c, c) for c in R.columns], dtype=object)
    A = R.to_numpy(float)
```

```
    fits = {r: M.fit_logit(A, y, ridge=r, iters=500)[0] for r in (ridge, *path)}
    w = fits[ridge]; p = M.predict_logit(w, A)
    X = np.column_stack([np.ones(n), A]); pen = np.r_[0.0, np.full(A.shape[1], ridge)]
    score = np.abs(X.T @ (y - p) - pen * w).max()
    moves = np.abs(fits[path[0]] - fits[path[1]]) > tol
    bread = np.linalg.inv((X * (p * (1 - p))[:, None]).T @ X + np.diag(pen))
    S = pd.DataFrame(X * (y - p)[:, None]).groupby(g).sum().to_numpy(); G = len(S)
    cov = bread @ (G / (G - 1) * S.T @ S) @ bread
    coef, se = np.where(moves, np.nan, w), np.where(moves, np.nan, np.sqrt(np.diag(cov)))
    note = ["not estimable: separation (the estimate grows without bound as the ridge shrinks)" if m
            else "identical columns in the data: one combined term" if t in names.values() else "" for t, m in
     zip(terms, moves)]
    t = pd.DataFrame({"term": terms, "coef": coef, "se_cluster_event": se, "z": coef / se, "estimable": ~moves,
     "note": note})
    idx = np.where(moves)[0]
    if len(idx) >= 2:
        a = np.zeros(len(w)); a[idx] = 1
        if abs(a @ fits[path[0]] - a @ fits[path[1]]) <= tol:
            v = np.sqrt(a @ cov @ a)
            t = pd.concat([t, pd.DataFrame([dict(term="sum: " + " + ".join(terms[idx]), coef=a @ w,
     se_cluster_event=v, z=a @ w / v,
                                                 estimable=True, note="sum of the separated terms, which the
     data do identify")])],
                          ignore_index=True)
    t.attrs["max_abs_score"] = score
    return t, p
```

Code 41: Variance inflation factors (scripts/part1_models.py)

```
def vif(D, tol=1e-10):
    """Variance inflation factor of each column: 1 / (1 - R^2) from regressing it on all the others with an
    intercept. A column the others reproduce exactly (residual sum of squares below tol times its total) is
    reported as infinite rather than hidden by a pseudo-inverse."""
    A = D.to_numpy(float); n = len(A); out = []
    for j in range(A.shape[1]):
        v = A[:, j]; X = np.column_stack([np.ones(n), np.delete(A, j, axis=1)])
        rss = np.sum((v - X @ np.linalg.lstsq(X, v, rcond=None)[0]) ** 2); tss = np.sum((v - v.mean()) ** 2)
        out.append(np.nan if tss == 0 else np.inf if rss <= tol * tss else tss / rss)
    return pd.DataFrame({"term": D.columns, "vif": out}).sort_values("vif", ascending=False)
```

Code 42: Calibration by decile (scripts/part1_models.py)

```
def calibration(prob, y, bins=10):
    q = pd.qcut(prob, bins, labels=False, duplicates="drop")
    return pd.DataFrame({"decile": q, "p": prob, "y": y}).groupby("decile").agg(mean_predicted=("p", "mean"),
                                                                                observed_rate=("y", "mean"), dyads
     =("y", "size")).reset_index()
```